

SCS 1301
Data Structures and Program Design Using C
AY 23 – Semester 01

Exercise V: File Handling in C

Dilanga Harshani

1. Write C programs to perform the following file operations.

- (a) Write a program that asks the user for a filename and checks whether it exists. If it does, read and display its entire content on the screen one character at a time, using a loop. If the file does not exist, print an appropriate message instead of terminating abnormally.
- (b) Write a program that asks the user for a filename. If the file does not already exist, create it, prompt the user to type a single line of text, and write that line to the new file. If the file already exists, do not modify it; instead, print a message informing the user that the file is already present.
- (c) Write a program that reads a text file one line at a time and displays each line on the screen with its line number placed in front of it, for example:

<pre>1: This is the first line 2: This is the second line</pre>

- (d) Write a program that asks the user how many student records are to be entered. For each student, read a roll number, a name, and the marks obtained, and write all three values as a single record to a file. The record should combine an integer, a string, and a floating point value in one line of output. Once all records have been written, reopen the file and read every record back, displaying the data as a table on the screen.
2. The following exercises require your program to add information to a file while leaving its existing content untouched.
- (a) A department keeps an attendance log in a single file. Every time your program is executed, it should ask for a student's name and add one new entry to the log, consisting of a serial number followed by the name. The serial number of the new entry must continue on from whatever the last entry in the file already was; for example, if the file currently ends with entry number 7, the entry your program adds must be numbered 8. Your program must therefore determine the last serial number present in the file before deciding what to write, and it must do so without erasing any of the entries already recorded.
 - (b) Extend the program in part (a) so that, once the new entry has been added, the complete log is displayed on the screen in full.
3. The following exercises require modifying the content of an existing file at a specific location, rather than at its beginning or its end.

- (a) Write a program that asks the user for the name of an existing file, a byte position within that file, and a string. The program should replace the file's content starting at the given position with the supplied string, without affecting the bytes that come before that position or the bytes well beyond the replaced region. Display the file's content before and after the change to demonstrate that the rest of the file is unaffected.
 - (b) Positioning the file pointer and writing directly at that position, as in part (a), always overwrites whatever content already occupied that location. Write a program that instead inserts a given string at a chosen position in an existing file such that none of the original content is lost; the content that originally began at that position must still be present in the file, appearing immediately after the inserted string rather than being replaced by it. Include, as a comment in your submitted file, a short explanation of why positioning and writing directly is not sufficient for this task, and describe the additional step your program takes to preserve the original content.
 - (c) Test the program from part (b) using a string to insert that is longer than the space you set aside to hold the preserved content. Include a comment in your submitted file describing what you observe and whether your program still behaves correctly under this condition.
4. The following exercises use file handling to build small text utilities.
- (a) Write a program that duplicates the entire content of one text file into a second file, with both filenames supplied by the user.
 - (b) Write a program that determines the number of lines, words, and characters present in a given text file, similar in purpose to the Unix `wc` command.
 - (c) Write a program that searches a text file for a word supplied by the user and reports the total number of occurrences found, together with the line number on which each occurrence appears.
 - (d) Write a program that produces a new file containing the lines of a given file in reverse order, so that the last line of the original file becomes the first line of the new file.
5. The following exercises describe file processing procedures in full. Each specifies exactly what your program is expected to do; you are not expected to know any technique beyond what is described below.
- (a) Two files each contain a list of integers, one integer per line, with the integers in each individual file already arranged in increasing order. Write a program that reads both files and produces a third file containing every integer from both files, still arranged in increasing order. Your program must build this combined list by repeatedly comparing the next unread integer from each file and writing the smaller of the two to the output file, then advancing only through the file that the value was taken from. Do not read both files completely into memory and sort the combined result as a separate step; the output must be produced by this comparison procedure alone.
 - (b) Write a program that compares two text files line by line and reports the number of the first line at which their content differs. If every line up to the end of the shorter file is identical between the two files, report that the shorter file ends before any difference was found. If the two files are identical in their entirety, report that no difference exists.
 - (c) A simple way of disguising a piece of text is to shift every alphabetic character forward by a fixed number of positions in the alphabet, wrapping around from 'z' back to 'a' when

necessary. For example, with a shift of three, 'a' becomes 'd', 'y' becomes 'b', and 'z' becomes 'c'. Characters that are not letters are left unchanged, and the case of each letter (upper or lower) is preserved. Write a program that reads a text file character by character, applies this shift with a shift amount supplied by the user, and writes the result to a new file. Then write a second program that reverses this process, restoring the original file when given the same shift amount that was used to disguise it.

6. Write a program that opens a file in a mode permitting both reading and writing, and takes three values from the user at runtime: a string to write to the file, a byte position within the file, and a second string to write starting at that position, overwriting whatever was previously there. After performing both writes, your program should determine and display, without hardcoding any position or relying on the specific values used in a single run:
 - (i) the character currently located exactly two bytes before the file's current position,
 - (ii) the current byte offset within the file, measured from the beginning.

Run your program with at least three different combinations of input strings and positions, and include the inputs and the resulting output for each run as a comment in your submitted file. Then explain, in one or two sentences added as a comment, why the byte offset reported in part (ii) is not simply equal to the length of the first string written.

7. So far, every file you have handled has been opened in text mode, using mode strings such as "r", "w", and "a". C also allows a file to be opened in binary mode, by appending the letter b to the mode string, giving "rb", "wb", and so on. Binary mode simply tells the library to transfer the bytes of a file exactly as they are, without any of the line ending translation that text mode may otherwise perform. The following exercises examine file handling on files that do not contain plain text. The C library places no restriction on what a file may contain; whether a file is treated as text is a decision made by the programmer, not something enforced by the language.
 - (a) Obtain a small image file, such as a .jpg or .png. Write a program that opens it in text mode and reads it using a line oriented input function, in a loop, printing whatever is returned to the screen. Run the program and, as a comment in your submitted file, explain what you observe and why a line oriented function of this kind does not process an image file in a meaningful way.
 - (b) Write a program that copies the same image file using binary mode and a character oriented loop, first declaring the variable used to hold each byte as `char`. Run the program and compare the size of the copy against the size of the original, for example using `ls -l` on both files. Then change the type of that variable to `int` only, and repeat the comparison. As a comment in your submitted file, report the sizes observed in both cases and explain the difference, with reference to the value of `EOF` and what happens when a byte from the image file coincides with that value once narrowed into a `char`.
